Team Work Breakdown: Member-by-Member Build Report

Version 1.0 · Companion to the System Architecture document

1. How to use this report

This report gives each of the seven team members a complete brief for their module: what to build, the rules it must follow, the data it touches, the functions other people will call, and how to know it is finished. . Member 7 owns the foundation everyone else builds on, so their first days are the most urgent on the project.

Member	Member name	Module
1	Debojit Ghosh	Registration and RSVP
2	Niyam Ul Haque	Event Check-in and QR tickets
3	@+91 84347 84393	Analytics
4	Aritra Giri	Events (page builder and calendar)
5	Utsa paul	Sharing and invitations
6	Rajashree	Admin panel
7	Soham	Cross cutting (Auth, Notifications and project foundation)

2. Team-wide agreements

- Layering: pages, server actions and route handlers call services in `lib/services`; services call the database through `lib/db`. React components never touch the database.
- Every server action and route handler validates input with Zod and checks access with `requireRole` plus an ownership check inside the service. Never trust the client for role, owner or status.

- Schema: Member 7 merges the full schema from the architecture document in the first days. After that, each owner changes only their own tables, through a pull request that the whole team can see.
- Conventions: UUID primary keys, UTC timestamps plus an event timezone, typed results (success or a named error) from services, no database calls inside components.
- Pull requests: small and frequent, with a preview deployment per PR. Each tough module gets a reviewer from an easy module. The main branch must always deploy.
- Seed data: Member 7 owns the seed script; every member adds fixtures for their module so nobody waits on anybody else to test.
- Definition of done for any module: routes work for the right roles only, inputs are validated, empty and error states exist, mobile layout works, tests listed in the member's section pass, and the module is demonstrated end to end on a preview deployment.

2.1 Interface contract

Agree these names before building. Until the real function exists, callers use a stub that returns fake data.

Function or component	Owner	Used by
`requireRole(roles)`, `getCurrentUser()`	Member 7	Everyone
`sendNotification(type, registrationId)`, `sendInvitation(eventId, email)`	Member 7	Members 1, 5, 6
`getEventBySlug`, `getEventById`, `canManageEvent(userId, eventId)`, `listPublishedEvents`, `getQuestions(eventId)`	Member 4	Members 1, 2, 3, 5, 6
`EventPage` component	Member 4	Public page and preview
`<TrackView eventId />` embed	Member 3	Member 4 (placed on the event page)
`createTicketToken()`, `verifyTicketToken()`, `renderTicketQr()`	Member 2	Member 1

Function or component	Owner	Used by
`registerForEvent`, `cancelRegistration`, `approveRegistration`, `declineRegistration`, `listRegistrations(eventId)`	Member 1	Members 2, 3, 4, 7
`getFirstTouchSource()`	Member 3	Member 1
`getEventAnalytics(eventId)`	Member 3	Members 4, 6
`buildEventUrl(slug, ref)`, `generateEventQr(url)`	Member 5	Member 4
`getSetting(key)`	Member 6	Members 4, 5

3. Member 1: Registration and RSVP (Debojit Ghosh)

Mission: let attendees register for an event under capacity, approval and waitlist rules without ever overbooking, and give them a ticket.

Routes: `/e/[slug]/register`, `/my/registrations`, `/my/tickets/[id]`, `/dashboard/events/[id]/registrations`.

What to build

1. Registration form that renders the event's custom questions (text, select, checkbox) with required-field validation. Build the Zod schema from the questions at request time.
2. `registerForEvent`: open a transaction, lock the event row, check for an existing registration, count confirmed registrations, choose the status, insert the registration with a ticket token and the captured source, commit.
3. A pure function `decideInitialStatus(event, confirmedCount)` that implements the state machine, so it can be unit tested without a database.
4. Cancellation with automatic waitlist promotion (oldest first) in one transaction.
5. Organizer registrations page: list with status filters, approve and decline (single and bulk), and a seat summary. Approval re-checks capacity.

6. RSVP confirmation screen, the attendee's registrations list, and the ticket page showing event details and the QR image.
7. Calls to Member 7's `sendNotification` after each commit: confirmation, approved, declined, waitlisted, promoted from waitlist.
8. Optional: CSV export of registrations.

Rules and requirements

- State machine: pending, confirmed, waitlisted, declined, cancelled. New registration with approval required is pending; with a free seat it is confirmed; when full it is waitlisted if the waitlist is on, otherwise rejected with no row created. Seats taken equals confirmed registrations.
- The attendee must be signed in. One registration per user per event, enforced by a unique constraint; a double submit returns the existing registration.
- Registration is allowed only for published events that have not ended.
- If the organizer raises capacity, promote waitlisted people into the new seats. If capacity is lowered below the confirmed count, keep existing confirmed registrations and only block new ones.
- Use the pooled WebSocket Neon driver for the transaction; the HTTP driver cannot run interactive transactions.
- Capture the source with Member 3's `getFirstTouchSource()` and store it on the registration.

Data

Owns `registrations`. Reads `events`, `questions`, `users`.

Depends on and provides

Needs Member 7 (session, roles, notifications), Member 4 (event and question data) and Member 2 (token and QR image). Until Member 2 delivers, use a stub that returns a random string. Provides confirmed registrations to Member 2, counts and sources to Member 3.

Definition of done

- Unit tests cover every transition of the state machine.

- Concurrency test: 50 parallel registrations on an event with capacity 5 end with exactly 5 confirmed.
- Cancelling a confirmed registration promotes exactly one waitlisted person.
- An attendee cannot see or cancel another person's registration; an organizer can manage only their own events.

4. Member 2: Event Check-in and QR tickets (Niyam Ul Haque)

Mission: turn a confirmed registration into a verifiable QR ticket, and let staff check people in quickly and reliably on event day.

Routes: `/dashboard/events/[id]/checkin`, `POST /api/checkin`, `GET /api/events/[id]/live-count`.

What to build

1. Ticket token library (`lib/qr/token.ts`). A token is a random identifier of at least 128 bits plus an HMAC signature made with `QR_SIGNING_SECRET`. `verifyTicketToken` checks the signature first, using a constant-time comparison, so garbage is rejected without touching the database. Deliver this in the first days because Member 1 needs it.
2. `renderTicketQr(token)`: returns SVG or a PNG data URL using the `qrcode` package.
3. Scanner page: camera scanning with `html5-qrcode` (or similar), loaded lazily; start and stop controls; a clear message if camera permission is denied; a full-screen result banner (green for checked in, amber for already checked in with the original time, red for invalid, wrong event or not confirmed); ignore repeat reads of the same code for a few seconds.
4. `checkInByToken(token, eventId, staffUserId)` and `checkInManually(registrationId, staffUserId)`, both writing one attendance row.
5. Manual check-in screen: search by name or email, filter, one-tap check-in. Optional: undo for organizers.
6. Live count: the endpoint returns checked in, confirmed total and remaining; the dashboard widget polls every 5 seconds and shows the most recent check-ins.

Rules and requirements

- Access: the organizer, co-hosts with `can_scan`, and admins. Everyone else gets a 403.
- Only confirmed registrations can check in, and the token must belong to this event.

- Scans are idempotent through the unique constraint on `attendance.registration_id`.
- Decide and document the check-in window; recommended is the day of the event.
- The camera needs HTTPS, so test on a Vercel preview URL rather than a local network address.
- Manual check-in must always work, even if the camera does not.

Data

Owns `attendance`. Reads `registrations`, `events`, `event_hosts`.

Depends on and provides

Needs Member 7 (auth), Member 4 (`event_hosts` and `canManageEvent`) and Member 1 (confirmed registrations; use seed rows until ready). Provides the token library to Member 1 and attendance data to Member 3.

Definition of done

- Token tests: valid, tampered, truncated, wrong secret and wrong event all behave correctly.
- Double scan, cancelled registration and unconfirmed registration are handled.
- Scanner verified on at least one iPhone (Safari) and one Android phone (Chrome).
- Live count matches the attendance table exactly.

5. Member 3: Analytics (@+91 84347 84393)

Mission: record how people find and use each event, and show organizers numbers they can trust.

Routes: `/dashboard/events/[id]/analytics`, `POST /api/track`.

What to build

1. `<TrackView eventId />` client component. On load it posts to `/api/track`; Member 4 places it on the public event page.
2. The tracking endpoint: validate input, drop known bots by user agent, skip duplicates within a session, and write a `page_views` row with a visitor hash. The hash is built from a daily-rotating salt plus IP and user agent; never store the raw IP.

3. Source capture: read `ref` or `utm_source` from the URL, or the referrer host, and save it in a first-touch cookie. Expose `getFirstTouchSource()` so Member 1 can copy it onto the registration.
4. `getEventAnalytics(eventId)`: total views, unique daily visitors, registrations by status, views and registrations by source, conversion rate, attendance, and no-show rate.
5. Dashboard UI: stat cards, views over time, registrations over time, a sources table or bar chart, and a funnel from views to registrations to attendance. Use any lightweight chart library.
6. Optional: CSV export and a platform-wide summary for the admin panel.

Definitions (write these on the page)

- Unique views: the sum over days of distinct visitor hashes.
- Registration source: the first-touch source captured before the person registered.
- Attendance: the number of attendance rows.
- No-show rate: (confirmed registrations minus checked in) divided by confirmed registrations. Show pending until the event has ended.

Rules and requirements

- Bucket days in the event's timezone. Handle events with no data gracefully.
- Only the organizer, co-hosts and admins can view an event's analytics.
- Totals must reconcile with the source tables; write a test that compares dashboard numbers with direct counts.

Data

Owns `page_views`. Reads `registrations`, `attendance`, `events`.

Depends on and provides

Needs Member 4 (embeds the tracker), Member 1 (registration source and status), Member 2 (attendance) and Member 7 (auth). Provides `getFirstTouchSource()` to Member 1 and summaries to Members 4 and 6.

Definition of done

- Bot filtering, session deduplication and the no-show maths are unit tested.

- A script generates realistic page views so charts can be checked before real traffic exists.
- Counts on the dashboard match direct database counts.

6. Member 4: Events, page builder and calendar (Aritra Giri)

Mission: let organizers build and publish an event page, and let everyone browse events.

Routes: `/e/[slug]`, `/events`, `/dashboard/events`, `/dashboard/events/new`, `/dashboard/events/[id]/edit`, `/dashboard/events/[id]/preview`.

What to build

1. Page builder form: title; description in Markdown with a live preview; cover image and branding (accent colour, logo) uploaded straight to Vercel Blob using a server-issued token that limits type and size; start and end time with timezone; mode (in person, online, hybrid) with venue or online link; hosts and co-hosts (add by email, with a scan permission toggle); custom URL with a live availability check; capacity, approval and waitlist settings; custom registration question editor (add, reorder, remove; text, select, checkbox).
2. The shared `EventPage` component used by both the public route and the preview, mobile first, with states for draft preview, published, and ended.
3. Lifecycle actions: publish (with validation), unpublish, archive, and cache invalidation on every change.
4. Organizer dashboard list: status, registration count and quick links to registrations, check-in, analytics and share.
5. Calendar at `/events`: upcoming and past tabs, pagination, cards with cover, date, location and host. Exclude drafts and moderated-out events.
6. A register panel on the event page that shows the right state: register, registered, waitlisted, full, or ended. It links to Member 1′s flow.
7. Page metadata for search and sharing; place Member 3′s tracker on the page.

Rules and requirements

- Slugs use lowercase letters, digits and hyphens, 3 to 60 characters, unique regardless of case, with reserved words blocked (read the list from Member 6′s settings).
- End time must be after start time. Capacity cannot be set below the current confirmed count.

- Only the owner, co-hosts and admins can edit. Events with registrations are archived, never deleted.
- Sanitize rendered Markdown; no raw HTML.

Data

Owns `events`, `event_hosts`, `questions`.

Depends on and provides

Needs Member 7 (auth) and Blob configuration. Everyone depends on Member 4's read functions, so deliver those in the first days, even before the builder UI is finished.

Definition of done

- An organizer can go from a blank form to a live public page and see it in the calendar.
- Preview and public page look identical.
- Slug rules and edit permissions are tested.

7. Member 5: Sharing and invitations (Utsa paul)

Mission: make it effortless for organizers to promote an event and for shared links to look good.

Routes: `/dashboard/events/[id]/share`, `/e/[slug]/opengraph-image`, `/api/events/[id]/qr`.

What to build

1. Share page for organizers: copy link, per-channel links (WhatsApp, X, LinkedIn, Facebook, email, direct), and a Web Share button on mobile. Each link carries a `ref` parameter naming the channel.
2. Pure helper functions that build each platform's share URL with correct encoding, plus `buildEventUrl(slug, ref)`.
3. Dynamic Open Graph image with `next/og`, using the title, date, location and cover image, with a fallback design when there is no cover.
4. Event QR code: `generateEventQr(url)` returns SVG or PNG; a download button with size options and high error correction. Optional: a print-friendly poster page.

5. Invitations: paste or upload a list of emails, validate and deduplicate, preview the message, enforce the daily limit from platform settings, and send through Member 7's `sendInvitation`. Invitation links use `ref=invite`.

Rules and requirements

- Share links work only for published events; show a warning for drafts.
- Do not email the same address twice for the same event.
- Invitations need a small `invitations` table (event_id, email, status, sent_at, sent_by). Agree this with Member 7 and add it to the schema.
- Rate limit invitation sending per organizer.

Data

Owns `invitations`. Reads `events`.

Depends on and provides

Needs Member 4 (event data), Member 7 (email sending) and Member 6 (daily limit setting). Provides URL and QR helpers that Member 4 can reuse on the event page.

Definition of done

- Link previews render correctly when tested on WhatsApp and LinkedIn.
- The event QR code scans to the right page on both phone types.
- Duplicate and invalid emails are rejected with clear messages.

8. Member 6: Admin panel (Rajashree)

Mission: give admins the tools to manage users, organizers and events, moderate content, and configure the platform.

Routes: `/admin`, `/admin/users`, `/admin/organizers`, `/admin/events`, `/admin/moderation`, `/admin/settings`.

What to build

1. Admin layout and overview with counts of users, organizers, events and flagged items.
2. Users: searchable, paginated table with role and status filters; change role; suspend and restore; a detail view showing the person's registrations and events.

3. Organizers: list with event counts; approve and revoke organizer upgrades if the team chooses an approval flow.
4. Events: all events with filters (status, moderation status, organizer); force unpublish, archive, restore; link to the public page.
5. Moderation: a queue of flagged events with actions to dismiss, remove (sets the event to removed and unpublishes it) and restore. A reason is required, and the organizer is notified by email through Member 7.
6. Platform settings: site name, default timezone, reserved slugs, daily invitation limit, and a global registration toggle. Provide a typed `getSetting(key)` with defaults and caching.
7. Audit trail: every admin action writes a `moderation_actions` row with who, what, target and reason.

Rules and requirements

- All admin routes are protected by middleware and again by `requireRole('admin')` inside every action.
- An admin cannot suspend or demote themselves, and the last admin cannot be removed.
- Destructive actions ask for confirmation.
- Build a reusable data table component (search, filters, pagination) and share it with the rest of the team.

Data

Owns `moderation_actions` and `platform_settings`. Reads `users` and `events`.

Depends on and provides

Needs Member 7 (roles) and Member 4 (event data). Provides `getSetting` to Members 4 and 5.

Definition of done

- A non-admin gets a 403 on every admin route and action.
- Suspending a user takes effect on their next protected action.
- Every action appears in the audit trail.

9. Member 7: Auth, Notifications and foundation (Soham)

Mission: set up the project everyone builds on, then deliver sign-in, roles and email.

Routes: `/login`, `/signup`, `/api/auth/[...nextauth]`, `/api/cron/reminders`, `/api/health`, plus `middleware.ts`.

What to build, in order

1. Foundation (first days): Next.js repo with Tailwind, linting and an environment example file; Neon connection with a pooled driver and Drizzle; the full schema and migrations from the architecture document (plus `invitations`); a seed script; Vercel project with preview deployments; Blob token; email provider account and domain records.
2. Auth.js: Google sign-in and email plus password, hashed passwords, signup with validation, JWT session carrying user id, role and status, and an edge-safe configuration for middleware.
3. Access control: middleware protecting `/dashboard`, `/admin` and `/my`; `requireRole` and `getCurrentUser` helpers; suspended accounts blocked at the service layer; organizer upgrade request flow; a minimal profile page; role-aware navigation.
4. Notifications: email client, templates (confirmation with a link to the ticket, approved, declined, waitlisted, promoted from waitlist, reminder, invitation, moderation notice), plain-text fallbacks, and `sendNotification` that writes `notifications_log` with a unique key on registration and type.
5. Cron: `/api/cron/reminders` guarded by `CRON_SECRET` sends reminders to confirmed registrations for events starting inside the window and retries failed sends; schedule it in `vercel.json` (hourly if the plan allows, otherwise daily).
6. A README that lets a new teammate run the project in minutes.

Rules and requirements

- Normalize emails to lowercase. Enforce a minimum password length. Rate limit login.
- Never commit secrets; keep separate values per environment.
- Never email cancelled or declined registrations. Sending must be idempotent.
- Send confirmations after the registration transaction commits so users are not kept waiting.

Data

Owns `users` and `notifications_log`, plus the schema and migrations overall.

Depends on and provides

Depends on nothing at the start, and everyone depends on step 1 and step 3. Once the foundation is stable, Member 7 should help with Check-in or Analytics.

Definition of done

- A new teammate can clone, configure and run the app using the README alone.
- Roles redirect correctly: attendees cannot reach `/dashboard` or `/admin`, organizers cannot reach `/admin`.
- A confirmation email arrives in a real inbox and is not marked as spam.
- The reminder endpoint rejects requests without the cron secret and never sends a reminder twice.

10. Who helps whom

Easy-module members will finish earlier, so plan this ahead of time:

- Member 4 supports Member 1 with form and ticket UI, and with the register panel on the event page.
- Member 7 supports Member 2 with scanner testing, or Member 3 with tracking and cleanup jobs.
- Member 5 supports Member 3 with chart components and the sources view.
- Member 6 supports everyone with the shared data table, test data and the end-to-end test.

Weekly checkpoint: each member demos their module on the preview deployment, flags blockers on the interface contract, and updates the seed data.